AMSS 2026 — Lecture 10: Cross-Layer Traceability

Traian-Florin Șerbănuță

2026

Today's Agenda

1. Frame: the links between the artifacts
2. The full trace, end to end
3. Live opener: drive AI to a trace — watch a link break
4. Reading critically: finding broken links
5. How to audit a trace
6. Bridge to W11 + Lab 5 this week

Recap: Architect + Critic

- **Architect / director (B):** drive AI through the SDLC.
- **Critic / reviewer (A):** read AI's output and name what's wrong or missing.

In W2-W9 you critiqued individual artifacts. Today: the **links between them** — does the whole chain hold together?

Why Traceability

- **AI builds each layer plausibly — and independently.** The class diagram looks right; the tests look right.
- **But they drift.** The test checks a flat fare; the requirement said per-minute. Nobody linked them.

Today's question: **does every requirement reach a test — and every artifact trace back to a requirement?**

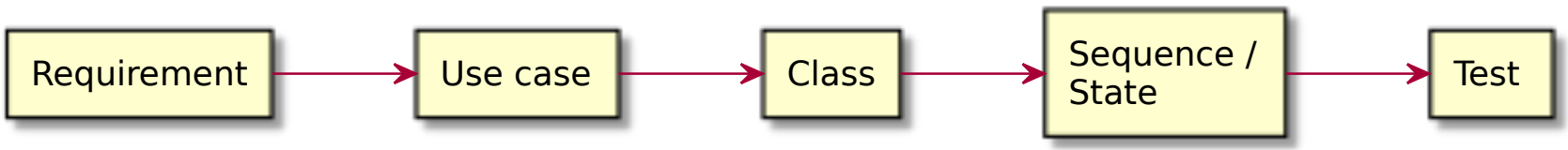
Literacy Floor: F1 + F3 + F4

From W1: in the oral defense, *unaided*, you must demonstrate:

- **F1** — read & critique AI-generated artifacts on the spot.
- **F3** — articulate why you directed AI a certain way.
- **F4** — defend traceability across your project.

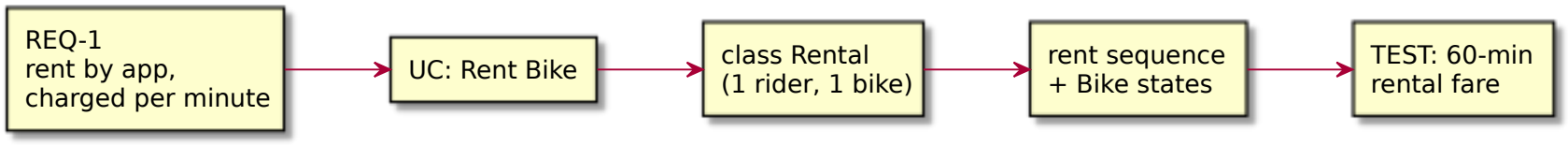
Today is **F4** at its sharpest: walk the trace from any requirement to its test and back, and name the broken links. This is the skill the whole course pointed toward.

The Full Trace



Every node is an artifact you have built since W2. One chain, requirement to test.

A Worked Trace: “Rent a Bike”



The bike-sharing chain, concrete: the requirement is realised by the use case, the class, the behaviour — and verified by the test.

Each Link Is a Claim

Every arrow asserts something:

- "this **use case** realises that requirement"
- "this **class** realises that use case"
- "this **test** verifies that requirement"

A link you **cannot justify** is a broken link — even if both ends are fine.

Conventions: IDs + Links

Make the links **explicit**, not implicit:

- Give artifacts IDs: `REQ-1`, `UC-2`, `TEST-3`.
- Tag each artifact with what it traces to: a test names its requirement.
- Keep a small **trace matrix**: requirement across, artifacts down.

If the link lives only in your head, it breaks silently.

Demo: Drive AI to a Trace

Live: ask AI for the full trace of a new feature. Walk it — does every layer agree, and does the test match the requirement?

Prompt to AI: *"For a new feature — a rider can reserve a bike for 15 minutes before pickup — give me the requirement, the use case, the class changes, the sequence, and a test."*

AI Builds Layers Independently — and They Drift

Each artifact can be locally fine and globally inconsistent. The critic question:

> *"Does every requirement reach a test — and does every artifact trace back to a requirement?"*

Defect #1: Orphan Requirement

`REQ-2: a rider can report a damaged bike` — but no use case, no class, no test realises it.

Critique: *"Walk REQ-2 forward. Where does it go? If nowhere, the feature was silently dropped."*

Defect #2: Orphan Artifact / Gold-Plating

A `LoyaltyPointsManager` class, or a test for surge-pricing — but **no requirement** asked for either.

Critique: *"Walk this backward. Which requirement needs it? If none, why is it here?"*

Defect #3: Dangling Link

The rent sequence sends `charge()` to a `Wallet` class — but `Wallet` **is not in the class diagram**.

Critique: *"This link points at something that doesn't exist. Add the class, or fix the message."*

Defect #4: Drift / Inconsistency

The class says `Payment`; the sequence says `Wallet`. The requirement says **per-minute**; the test checks a **flat fare**.

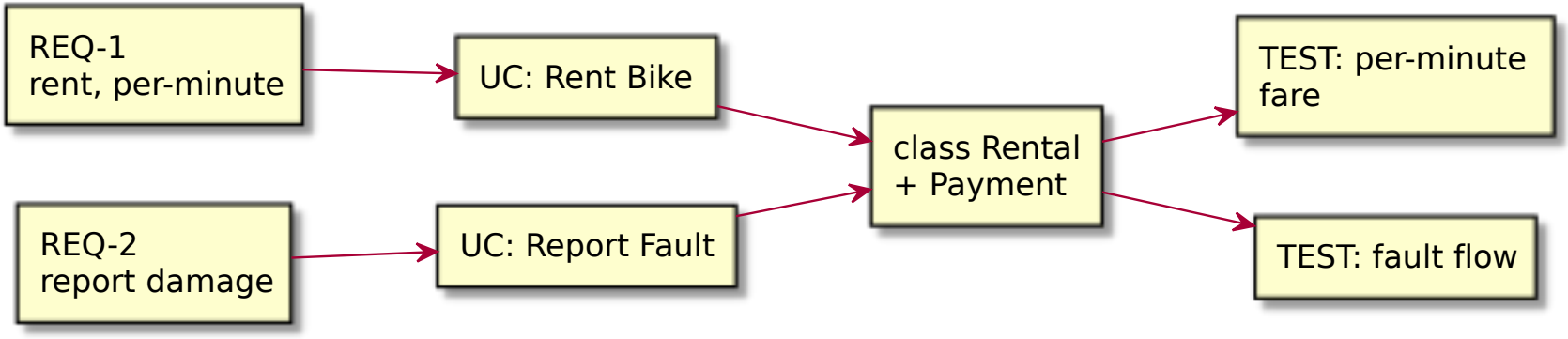
Critique: *"The layers disagree. Which one is right — and fix the others to match."*

Defect #5: Stale Link

The requirement changed — `per-minute` became `per-hour` — but the use case, class, and test still encode **per-minute**.

Critique: *"The trace points at the old requirement. A change in one node must propagate along every link."*

The Repaired Trace



Every requirement reaches a test; every artifact traces back; the layers agree.

How to Audit a Trace

A fixed order, every time:

1. Does every **requirement** reach a **test**? (forward walk)
2. Does every **artifact** trace back to a **requirement**? (backward walk)
3. Do the **layers agree**? (no drift across a link)
4. Any **stale links**? (did a change propagate?)

The Critique Loop on Traceability

walk the trace -> name the broken link -> fix the artifact / re-prompt -> re-walk.

Same architect-critic loop as W2-W9 — now on the chain itself.

The Human Owns the Trace

AI generates the layers. Keeping them consistent — walking the trace, catching the drift, propagating the changes — only the human does.

The trace is **the human's artifact**. It's what the oral defense checks, and AI can't maintain it for you.

This IS the Checkpoint Defense

Lab 5 this week: you walk **your project's** trace — requirement to test — in a short checkpoint defense.

Broken links are exactly what examiners probe: *"show me the test for this requirement"*; *"which requirement needs this class?"*

F4 — The Skill the Whole Course Built Toward

- W2 requirements, W4 classes, W6 sequences, W7 states, W3 tests — every week was a **node**.
- Today is the **chain**.

The oral defense is a trace walk. If you can walk it, you can defend it.

Next Week: Model Evaluation & Quality (W11)

Traceability checks the links hold. **W11** checks the quality of what they connect:

- consistency, completeness, correctness;
- static evaluation, conformance, simulation;
- and where **AI is a worse evaluator than you**.

F1 + F3 + F4 — The Through-Line

Today you drilled:

- **F4** — walked the trace requirement-to-test and named the broken links.
- **F1** — read across artifacts and caught drift no single artifact reveals.

F3: when you fix a broken link, you say *why* the layers must agree.

That's It For Today

- Next lecture (W11): model evaluation & quality.
- This week's lab (Lab 5): project workshop + checkpoint defense — walk your trace.

Questions?